Rational Matrix Policy Optimization for Transformer Language Models

Anonymous authors
Paper under double-blind review

Abstract

Transformer MLP sublayers contain large input and output matrices wrapped around a scalar nonlinearity. Standard optimizers usually update these parameters with one nearly uniform rule, even when the nonlinearity exposes layerwise and groupwise structure. We propose Rational Latent Basis (RLB), a global-rational Transformer sublayer whose grouped rational curves make this structure explicit, and MatrixPolicy, an optimizer that uses the resulting matrix roles, gain statistics, relative update pressures, and positive scale gauge. The main experimental configuration uses the global-rational/no-local-atom RLB variant: a single-branch grouped P5/Q4 rational sublayer with no local atom path and no atom parameter group. MatrixPolicy applies a centered group-gradient policy, a role/depth matrix update with a short early matrix-normalized branch, and a bounded post-step gauge rebalance while leaving non-matrix parameters on AdamW.

1 Method

This section defines the method used in the E1 and E2 experiments. The method has two coupled parts. The activation part is a global-rational RLB sublayer that replaces the usual Transformer MLP sublayer. The optimizer part is MatrixPolicy, which updates the RLB input and output matrices with rules that use the RLB geometry. The activation alone is not the claim: matched RLB controls with the same global-rational sublayer and different optimizers are required to isolate the MatrixPolicy contribution.

1.1 Global-Rational RLB Sublayer

Consider Transformer layer $l \in \{1, \dots, L\}$ with MLP-sublayer input $x_l \in \mathbb{R}^d$. RLB uses a single projected hidden representation of width $H = Gm$, partitioned into G groups of width m:

$$z_l = A_l x_l, \qquad z_l = \operatorname{concat}_{g=1}^{G} z_{l,g}, \qquad z_{l,g} \in \mathbb{R}^m, \tag{1}$$

where $A_l \in \mathbb{R}^{H \times d}$ is the input matrix. Each group is normalized by its RMS radius,

$$r_{l,g} = \sqrt{m^{-1}\|z_{l,g}\|_2^2 + \epsilon_{\mathrm{rms}}}, \qquad u_{l,g} = z_{l,g}/r_{l,g}. \tag{2}$$

The scalar rational curve $R_{l,g}$ is applied coordinatewise to $u_{l,g}$ and then multiplied by the same radius:

$$h_{l,g} = r_{l,g} R_{l,g}(u_{l,g}), \qquad h_l = \operatorname{concat}_{g=1}^{G} h_{l,g}, \qquad y_l = B_l h_l, \tag{3}$$

where $B_l \in \mathbb{R}^{d \times H}$ is the output matrix and $\epsilon_{\mathrm{rms}} > 0$ is the RMS floor. The output y_l is inserted in the Transformer block at the same location as the usual MLP-sublayer output.

The main RLB variant is global rational. For each layer and group,

$$R_{l,g}(u) = \frac{P_{l,g}(u)}{Q_{l,g}(u)}, \qquad P_{l,g}(u) = \sum_{i=0}^{5} a_{l,g,i} u^i, \tag{4}$$

and

$$Q_{l,g}(u) = 1 + |b_{l,g,1}|\,|u| + |b_{l,g,2}|\,u^2 + |b_{l,g,3}|\,|u|^3 + |b_{l,g,4}|\,u^4. \tag{5}$$

The coefficients are initialized from a least-squares SiLU fit on a fixed probe interval and are then trained. Because every nonconstant denominator term in equation 5 is nonnegative on the real line, $Q_{l,g}(u) \geq 1$ for all real u. The curve is therefore pole-safe on real inputs. The absolute-value terms make it piecewise smooth, which is the intended numerical tradeoff: the denominator is expressive enough for a trainable rational response while avoiding real poles during language-model training.

Global rational means that the learned nonlinear response is the P5/Q4 curve itself. There are no local Cauchy atoms, no atom centers, no atom coefficients, and no atom parameter group in the main configuration. This naming is important for the experiments: both MatrixPolicy rows and non-MatrixPolicy RLB optimizer controls use the same global-rational/no-local-atom RLB source.

1.2 Optimizer-Visible RLB Structure

The definitions in equation 1 through equation 3 separate three functional roles. The matrix A_l selects the domains in which the rational curves operate. The coefficients of $R_{l,g}$ set the scalar response and local derivative profile inside each group. The matrix B_l recombines the rational features into the residual stream. MatrixPolicy uses this separation rather than treating all RLB parameters as one undifferentiated tensor family.

RLB also has a positive group gauge. Let $D_l(a) = \mathrm{diag}(a_1 I_m, \ldots, a_G I_m)$ for any positive vector $a \in \mathbb{R}^G_{>0}$, let $R_l = \{R_{l,g}\}_{g=1}^G$, and write $f_l(x; A_l, B_l, R_l)$ for the RLB block map defined by equation 1 through equation 3. If $\epsilon_{\mathrm{rms}} = 0$ and all group radii are nonzero, then

$$f_l(x; A_l, B_l, R_l) = f_l(x; D_l(a)A_l, B_l D_l(a)^{-1}, R_l). \tag{6}$$

The stabilized radius used in training makes this identity approximate near the RMS floor, but the identity remains the organizing geometry away from degenerate groups. MatrixPolicy therefore has two responsibilities: move the represented function and keep the chosen matrix representative from drifting along an uncontrolled scale direction.

The optimizer partition follows this structure. MatrixPolicy is applied to the RLB matrix set $\{A_l, B_l\}_{l=1}^L$. Non-matrix parameters, including embeddings, attention weights, normalization parameters, and the rational numerator and denominator coefficients, are updated by the AdamW child optimizer (Kingma & Ba, 2015; Loshchilov & Hutter, 2019). The separate rational coefficient optimizer and all transport-style coefficient metric branches are inactive in the main configuration.

1.3 On-Policy Signals

MatrixPolicy uses live statistics collected from each RLB layer and group before the child optimizer steps. Let $\epsilon_p > 0$ be the small floor used for pressure statistics. For group g in layer l, let $A_{l,g} \in \mathbb{R}^{m \times d}$ be the corresponding row block of A_l and let $B_{l,g} \in \mathbb{R}^{d \times m}$ be the corresponding column block of B_l. The relative matrix-gradient pressures are

$$p^{\mathrm{in}}_{l,g} = \frac{\mathrm{rms}(\nabla A_{l,g})}{\mathrm{rms}(A_{l,g}) + \epsilon_p}, \qquad p^{\mathrm{out}}_{l,g} = \frac{\mathrm{rms}(\nabla B_{l,g})}{\mathrm{rms}(B_{l,g}) + \epsilon_p}. \tag{7}$$

For the global-rational RLB variant, the rational activity statistic is the RMS of the numerator and denominator gradients in that group:

$$p^R_{l,g} = \left[\frac{1}{2}\left(\mathrm{rms}(\nabla a_{l,g,\cdot})^2 + \mathrm{rms}(\nabla b_{l,g,\cdot})^2\right) + \epsilon_p\right]^{1/2}. \tag{8}$$

The implementation generalizes this formula to local-atom ablations by including the active atom tensor, but that tensor is absent in the main method.

Let $q^{\text{in}}_{l,g}$, $q^{\text{out}}_{l,g}$, and $q^{R}_{l,g}$ be exponential moving averages of equation 7 and equation 8 with decay 0.95, floored by ϵ_p before logarithms. MatrixPolicy forms two log-domain signals:

$$\pi_{l,g} = \log q^{\text{in}}_{l,g} - \log q^{\text{out}}_{l,g}, \qquad \alpha_{l,g} = \log q^{R}_{l,g} - \frac{1}{2}\left(\log q^{\text{in}}_{l,g} + \log q^{\text{out}}_{l,g}\right). \tag{9}$$

The pressure $\pi_{l,g}$ measures imbalance between the two matrix roles. The activity $\alpha_{l,g}$ measures whether rational coefficients are receiving large gradients relative to the adjacent matrices.

Two gain statistics are also recorded from the current activation distribution. For normalized coordinates $u_{l,g}$ sampled from the recent forward pass,

$$d^{\text{live}}_{l,g} = \text{rms}(R'_{l,g}(u_{l,g})), \qquad o^{\text{live}}_{l,g} = \text{rms}(R_{l,g}(u_{l,g})). \tag{10}$$

These are not full Jacobians of the Transformer block. They are cheap function-space proxies: derivative gain is used for the input-domain matrix A_l, and output gain is used for the recombination matrix B_l.

1.4 Group-Gradient Policy

Let t be the optimizer step, let T be the planned number of training steps, and write $\xi_t = t/T$. The group policy turns on smoothly through

$$\phi_t = \text{smoothstep}(0.02, 0.30, \xi_t), \tag{11}$$

where $\text{smoothstep}(a, b, \xi)$ is the cubic smoothstep gate from 0 to 1 over $[a, b]$, and geomean_j denotes the geometric mean over groups. For matrix role $r \in \{\text{in}, \text{out}\}$, MatrixPolicy constructs a group multiplier

$$c_{l,g,r} = c^{\text{gain}}_{l,g,r} c^{\text{pressure}}_{l,g,r} c^{\text{activity}}_{l,g}. \tag{12}$$

For the input role, the gain term uses $d^{\text{live}}_{l,g}$; for the output role, it uses $o^{\text{live}}_{l,g}$:

$$c^{\text{gain}}_{l,g,\text{in}} = \left(\frac{\text{geomean}_j\, d^{\text{live}}_{l,j}}{d^{\text{live}}_{l,g}}\right)^{0.20\phi_t}, \qquad c^{\text{gain}}_{l,g,\text{out}} = \left(\frac{\text{geomean}_j\, o^{\text{live}}_{l,j}}{o^{\text{live}}_{l,g}}\right)^{0.20\phi_t}. \tag{13}$$

The pressure term uses the clipped pressure $\tilde{\pi}_{l,g} = \text{clip}(\pi_{l,g}, -1.50, 1.50)$:

$$c^{\text{pressure}}_{l,g,\text{in}} = \exp(-0.10\phi_t \tilde{\pi}_{l,g}), \qquad c^{\text{pressure}}_{l,g,\text{out}} = \exp(+0.10\phi_t \tilde{\pi}_{l,g}). \tag{14}$$

Large positive pressure means the input matrix has larger relative gradient pressure than the output matrix, so the policy reduces the input-side multiplier and increases the output-side multiplier. The rational-activity term damps both matrix roles when rational coefficients are unusually active:

$$c^{\text{activity}}_{l,g} = \exp\left[-0.20\phi_t\, \text{ReLU}\left(\frac{\alpha_{l,g} - 0.05}{0.45}\right)\right]. \tag{15}$$

The final multiplier is centered and clipped:

$$c_{l,g,r} \leftarrow \frac{c_{l,g,r}}{\text{geomean}_j\, c_{l,j,r}}, \qquad c_{l,g,r} \leftarrow \text{clip}(c_{l,g,r}, 0.75, 1.35). \tag{16}$$

The centering keeps the mean log multiplier at zero before clipping, so the rule redistributes matrix update budget across groups instead of changing the global learning rate. The clip limits the influence of noisy group statistics.

1.5 Role and Depth Matrix Update

After group-gradient scaling, each RLB matrix is stepped by a role/depth AdamW branch and, early in training, a matrix-normalized branch. Let $M_{l,\text{in}} = A_l$ and $M_{l,\text{out}} = B_l$. Define

normalized depth $\delta_l = (l-1)/(L-1)$ for $L > 1$, with $\delta_l = 1/2$ if there is only one layer. The role factors are

$$\rho_{\text{in}}(l) = \text{clip}(1 - 0.50(\delta_l - 0.5), 0.55, 1.40), \qquad \rho_{\text{out}}(l) = \text{clip}(1 + 1.00(\delta_l - 0.5), 0.55, 1.40). \tag{17}$$

The AdamW learning-rate multiplier for role r is

$$s_{l,r}^{\text{Adam}} = \text{clip}\left(3.0 \max\{0.10, 1 + 1.20(\rho_r(l) - 1)\}, 0.40, 4.0\right). \tag{18}$$

This gives earlier layers a mild bias toward input-domain selection and later layers a mild bias toward output recombination.

The matrix-normalized branch is active only near the beginning of training. Let

$$\psi_{l,t} = \text{clip}\left(\exp[-0.30\Psi_{l,t} - 0.65\Omega_{l,t}], 0.10, 1.15\right), \tag{19}$$

where

$$\Psi_{l,t} = \frac{1}{G}\sum_{g=1}^{G} |\text{clip}(\pi_{l,g}, -1.50, 1.50)|, \qquad \Omega_{l,t} = \frac{1}{G}\sum_{g=1}^{G} \text{ReLU}\left(\frac{\alpha_{l,g} - 0.05}{0.45}\right). \tag{20}$$

The mixture fraction for role r is

$$\mu_{l,r,t} = \text{clip}\left(0.75 \text{ smoothstep}(0.02, 0.12, \xi_t)[1 - \text{smoothstep}(0.20, 0.36, \xi_t)]\rho_r(l)\psi_{l,t}, 0, 0.75\right). \tag{21}$$

Let η_t be the base learning rate. MatrixPolicy implements the matrix step as two stateful child optimizer steps with temporary learning-rate multipliers:

$$\begin{aligned} \widetilde{M}_{l,r}^{t+1} &= \text{AdamW}_{\eta_t s_{l,r}^{\text{Adam}}(1-\mu_{l,r,t})}(M_{l,r}^t; \nabla M_{l,r}^t), \\ M_{l,r}^{t+1} &= \text{MatNorm}_{\eta_t \mu_{l,r,t}}(\widetilde{M}_{l,r}^{t+1}; \nabla M_{l,r}^t). \end{aligned} \tag{22}$$

Here MatNorm denotes the matrix-normalized optimizer branch with momentum and Newton-Schulz normalization steps. The step in equation 22 describes the operational composition of two stateful updates; it is not an additive closed-form optimizer. Once equation 21 is permanently zero for all RLB matrix groups, the zero-learning-rate matrix-normalized child step is skipped. This changes runtime only, because it removes a step that would otherwise apply exactly zero learning rate.

1.6 Bounded Positive-Gauge Rebalance

After the child optimizer steps, the MatrixPolicy wrapper applies a bounded move along the positive group gauge in equation 6. This operation uses probe-grid gains, not the live activation-distribution gains used by the group-gradient policy. On a fixed grid over the normalized coordinate, define

$$d_{l,g}^{\text{probe}} = \text{rms}(R'_{l,g}(u)), \qquad o_{l,g}^{\text{probe}} = \text{rms}(R_{l,g}(u)). \tag{23}$$

Let

$$\gamma_{l,g} = \log \text{rms}(A_{l,g}) - \log \text{rms}(B_{l,g}) \tag{24}$$

be the current log matrix ratio. The target log ratio is

$$\tau_{l,g} = \frac{1}{2}\left(\log d_{l,g}^{\text{probe}} - \log o_{l,g}^{\text{probe}}\right) + 0.25\,\bar{\pi}_{l,g}, \qquad \bar{\pi}_{l,g} = \text{clip}(\pi_{l,g}, -1.25, 1.25). \tag{25}$$

The first term is a gain-ratio target; the second term shifts the representative toward lower persistent matrix-pressure imbalance. Rational activity controls how strongly the move is applied:

$$a_{l,g}^{\text{gauge}} = \exp\left(\text{clip}(0.10\,\bar{\alpha}_{l,g}, \log 0.75, \log 1.25)\right), \qquad \bar{\alpha}_{l,g} = \text{clip}(\alpha_{l,g}, -1.25, 1.25). \tag{26}$$

The scheduled log-scale move is

$$\ell_{l,g,t} = \mathrm{clip}\left(\frac{1}{2}[\tau_{l,g} - \gamma_{l,g}]\, s_{l,t}\, a_{l,g}^{\mathrm{gauge}}, -0.030, 0.030\right), \tag{27}$$

where

$$s_{l,t} = 0.50\ \mathrm{smoothstep}(0.03, 0.35, \xi_t)\ \mathrm{clip}(1 + 0.15(\delta_l - 0.5), 0.75, 1.25). \tag{28}$$

The matrices are then rescaled by

$$A_{l,g} \leftarrow e^{\ell_{l,g,t}} A_{l,g}, \qquad B_{l,g} \leftarrow e^{-\ell_{l,g,t}} B_{l,g}. \tag{29}$$

When optimizer moment tensors have compatible shapes, their states are scaled covariantly with the same group factors. In the homogeneous-radius idealization, equation 29 changes only the matrix representative. With the RMS floor in equation 2, the move is no longer exactly function-preserving near degenerate groups, so the implementation clips every log move by 0.030.

Algorithm 1: MatrixPolicy step for global-rational RLB.

1. Record per-group pressure EMAs and rational-activity EMAs from the current gradients.
2. Update live RLB gain statistics from the recent normalized activation distribution.
3. Apply the centered, clipped group-gradient multiplier equation 16 to the RLB matrix gradients.
4. Step non-matrix parameters, including rational coefficients, with the AdamW child optimizer.
5. Step RLB matrices with the role/depth AdamW branch and the early matrix-normalized branch in equation 22.
6. Apply the bounded positive-gauge rebalance in equation 29.

References

Xiangning Chen, Chen Liang, Da Huang, Esteban Real, Kaiyuan Wang, Hieu Pham, Xuanyi Dong, Thang Lu, Cho-Jui Hsieh, Yifeng Lu, and Quoc V. Le. Symbolic Discovery of Optimization Algorithms. In Advances in Neural Information Processing Systems, 2023. URL https://papers.neurips.cc/paper_files/paper/2023/hash/9a39b4925e35cf447ccba8757137d84f-Abstract-Conference.html.

Aaron Defazio, Xingyu Yang, Ahmed Khaled, Konstantin Mishchenko, Harsh Mehta, and Ashok Cutkosky. The Road Less Scheduled. In Advances in Neural Information Processing Systems, 2024. URL https://neurips.cc/virtual/2024/poster/96925.

Diederik P. Kingma and Jimmy Ba. Adam: A Method for Stochastic Optimization. In International Conference on Learning Representations, 2015. URL https://mlanthology.org/iclr/2015/kingma2015iclr-adam/.

Jeffrey Li, Alex Fang, Georgios Smyrnis, Maor Ivgi, Matt Jordan, Samir Gadre, Hritik Bansal, Etash Guha, et al. DataComp-LM: In Search of the Next Generation of Training Sets for Language Models. In Advances in Neural Information Processing Systems, Datasets and Benchmarks Track, 2024. doi: 10.52202/079017-0455. URL https://proceedings.neurips.cc/paper_files/paper/2024/hash/19e4ea30dded58259665db375885e412-Abstract-Datasets_and_Benchmarks_Track.html.

Kaizhao Liang, Lizhang Chen, Bo Liu, and Qiang Liu. Cautious Optimizers: Improving Training with One Line of Code. In International Conference on Learning Representations, 2026. URL https://openreview.net/forum?id=zBPZeRjfgu.

Hong Liu, Zhiyuan Li, David Hall, Percy Liang, and Tengyu Ma. Sophia: A Scalable Stochastic Second-order Optimizer for Language Model Pre-training. In International Conference on Learning Representations, 2024. URL https://proceedings.iclr.cc/paper_files/paper/2024/hash/06960915ba8674c7a898ec0b472b80ff-Abstract-Conference.html.

Ilya Loshchilov and Frank Hutter. Decoupled Weight Decay Regularization. In International Conference on Learning Representations, 2019. URL https://openreview.net/forum?id=Bkg6RiCqY7.

Yang Luo, Xiaozhe Ren, Zangwei Zheng, Zhuo Jiang, Xin Jiang, and Yang You. CAME: Confidence-guided Adaptive Memory Efficient Optimization. In Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics, pp. 4442–4453, 2023. doi: 10.18653/v1/2023.acl-long.243. URL https://aclanthology.org/2023.acl-long.243/.

Matteo Pagliardini, Pierre Ablin, and David Grangier. The AdEMAMix Optimizer: Better, Faster, Older. In International Conference on Learning Representations, 2025. URL https://iclr.cc/virtual/2025/poster/28625.

Guilherme Penedo, Hynek Kydlicek, Loubna Ben Allal, Anton Lozhkov, Margaret Mitchell, Colin Raffel, Leandro von Werra, and Thomas Wolf. The FineWeb Datasets: Decanting the Web for the Finest Text Data at Scale. In Advances in Neural Information Processing Systems, Datasets and Benchmarks Track, 2024. doi: 10.52202/079017-0970. URL https://papers.nips.cc/paper/2024/hash/370df50ccfdf8bde18f8f9c2d9151bda-Abstract-Datasets_and_Benchmarks_Track.html.

Luca Soldaini, Rodney Kinney, Akshita Bhagia, Dustin Schwenk, David Atkinson, Russell Authur, Ben Bogin, Khyathi Chandu, et al. Dolma: An Open Corpus of Three Trillion Tokens for Language Model Pretraining Research. arXiv preprint arXiv:2402.00159, 2024. URL https://arxiv.org/abs/2402.00159.

Nikhil Vyas, Depen Morwani, Rosie Zhao, Itai Shapira, David Brandfonbrener, Lucas Janson, and Sham Kakade. SOAP: Improving and Stabilizing Shampoo using Adam for Language Modeling. In International Conference on Learning Representations, 2025. URL https://proceedings.iclr.cc/paper_files/paper/2025/hash/e988664070e9591f93fdcf605f7dc623-Abstract-Conference.html.

Kaiyue Wen, David Leo Wright Hall, Tengyu Ma, and Percy Liang. Fantastic Pretraining Optimizers and Where to Find Them. In International Conference on Learning Representations, 2026. URL https://openreview.net/forum?id=2J51qUZ0iG.

Yushun Zhang, Congliang Chen, Ziniu Li, Tian Ding, Chenwei Wu, Diederik P. Kingma, Yinyu Ye, Zhi-Quan Luo, and Ruoyu Sun. Adam-mini: Use Fewer Learning Rates To Gain More. In International Conference on Learning Representations, 2025. URL https://proceedings.iclr.cc/paper_files/paper/2025/hash/45ae878717399e6f62d57c65f052cd46-Abstract-Conference.html.

Jiawei Zhao, Zhenyu Zhang, Beidi Chen, Zhangyang Wang, Anima Anandkumar, and Yuandong Tian. GaLore: Memory-Efficient LLM Training by Gradient Low-Rank Projection. In International Conference on Machine Learning, 2024. URL https://openreview.net/forum?id=hYHsrKDiX7.

A Exact Properties of Global-Rational RLB

This appendix separates facts that follow directly from the parameterization from heuristics used by MatrixPolicy. The denominator is pole-safe on real inputs, and the RLB matrix pair has an exact positive gauge when the RMS floor is removed and group radii are nonzero. The gain, pressure, and activity signals are designed from these facts but are not presented as a convergence proof or a natural-gradient derivation.

A.1 Pole-Safe Denominator

For one layer and group, the global rational denominator is

$$Q(u) = 1 + |b_1|\,|u| + |b_2|u^2 + |b_3|\,|u|^3 + |b_4|u^4. \tag{30}$$

For every real u, all nonconstant terms are nonnegative, so $Q(u) \geq 1$. Thus $P(u)/Q(u)$ has no real pole. This property is independent of the signs of the raw denominator parameters. The only cost is differentiability at the points where the absolute-value terms change derivative, which the implementation handles with standard automatic differentiation.

A.2 Positive Gauge Identity

Set $\epsilon_{\text{rms}} = 0$ and consider one nonzero group. For a positive scalar $a > 0$,

$$r(az) = \sqrt{m^{-1}\|az\|_2^2} = ar(z), \qquad u(az) = \frac{az}{r(az)} = \frac{z}{r(z)} = u(z). \tag{31}$$

Therefore

$$h(az) = r(az)R(u(az)) = ar(z)R(u(z)) = ah(z). \tag{32}$$

If the matching output columns are scaled by a^{-1}, the group contribution to $B_l h_l$ is unchanged. Applying the same argument independently to every group proves equation 6.

A.3 Effect of the RMS Floor

With $\epsilon_{\text{rms}} > 0$, scaling a group by a gives

$$u_\epsilon(az) = \frac{az}{\sqrt{a^2 m^{-1}\|z\|_2^2 + \epsilon_{\text{rms}}}} = \kappa(a, z)u_\epsilon(z), \tag{33}$$

where

$$\kappa(a, z) = \frac{a\sqrt{m^{-1}\|z\|_2^2 + \epsilon_{\text{rms}}}}{\sqrt{a^2 m^{-1}\|z\|_2^2 + \epsilon_{\text{rms}}}}. \tag{34}$$

If $m^{-1}\|z\|_2^2 \gg \epsilon_{\text{rms}}$, then $\kappa(a, z) \approx 1$ and the homogeneous gauge is a good local model. Near the floor, $\kappa(a, z)$ can differ from 1, so the rebalance is only an approximate gauge move. This is why equation 27 clips the per-step log scale and why the paper reports matched training behavior rather than claiming exact function preservation under the stabilized radius.

B Design Rationale

B.1 Gain Proxies for Matrix Updates

The output matrix has a direct first-order effect

$$\delta y_{l,g} = \delta B_{l,g} h_{l,g}. \tag{35}$$

Since $h_{l,g} = r_{l,g}R_{l,g}(u_{l,g})$, the RMS of the scalar rational output is a cheap proxy for how strongly that group feeds the recombination matrix. This motivates using $o_{l,g}$ for output-side matrix scaling.

The input matrix changes $z_{l,g}$ before normalization and rational evaluation. For the homogeneous-radius case, the differential of $h(z) = r(z)R(z/r(z))$ has the schematic form

$$\mathrm{d}h = \mathrm{d}r\, R(u) + r\ \mathrm{diag}(R'(u))\,\mathrm{d}u. \tag{36}$$

The full Jacobian also contains the derivative of the RMS normalization and interactions with the rest of the Transformer block. MatrixPolicy deliberately does not estimate that full Jacobian. It uses the scalar derivative RMS $d_{l,g}$ as a stable, low-cost proxy for the input-side sensitivity of the rational branch. The centered inverse-gain rule in equation 13 follows from this proxy: large-gain groups receive smaller raw matrix-gradient scale, and small-gain groups receive larger scale, with the log mean recentered.

B.2 Pressure as Gauge-Imbalance Evidence

In the exact gauge model, the infinitesimal transform $A_{l,g} \mapsto e^{s} A_{l,g}$ and $B_{l,g} \mapsto e^{-s} B_{l,g}$ leaves the represented function unchanged. If no regularization, moment state, or RMS floor were present, the loss derivative along this direction would satisfy

$$\langle \nabla_{A_{l,g}} \mathcal{L}, A_{l,g} \rangle_F - \langle \nabla_{B_{l,g}} \mathcal{L}, B_{l,g} \rangle_F = 0. \tag{37}$$

The implementation uses the RMS ratio pressure in equation 7 instead of this signed inner product. The RMS ratio is less sensitive to cancellation and aligns with optimizer step size, because it measures gradient magnitude relative to parameter magnitude. The pressure $\pi_{l,g}$ is therefore an empirical gauge-imbalance signal, not an exact quotient-space gradient.

The same signal appears twice for different reasons. In equation 14, it redistributes the next matrix-gradient update between input and output roles. In equation 25, it shifts the post-step gauge target toward a representative with less persistent relative update imbalance. Both uses are bounded by clipping, so a noisy group cannot dominate either the gradient policy or the gauge move.

B.3 Early Matrix-Normalized Branch

The matrix-normalized branch is scheduled early because the RLB matrices define the domains and recombinations seen by the rational curves. Early in training, changing matrix directions can rapidly change which normalized coordinates each group sees. Later, after the rational curves, embeddings, attention layers, and output recombinations have co-adapted, a persistent normalized matrix branch adds runtime and can obscure the RLB-specific role/depth policy. MatrixPolicy therefore uses the branch as an early conditioning mechanism and then decays it to zero through equation 21.

This design also matches the empirical comparison boundary. The method is not "use Muon everywhere." The standalone Muon controls are separate baselines, while MatrixPolicy restricts the matrix-normalized branch to RLB matrices, modulates it by RLB pressure and activity statistics, and removes it after the early window.

B.4 Global Rational Instead of Local Atoms

The main paper method removes the local atom path from RLB. A priori, the global-rational variant is the cleaner object for an optimizer paper: each group has one trainable rational curve with numerator and denominator parameters, and MatrixPolicy can be described through matrix roles, rational gains, pressure statistics, and gauge balance. There is no separate atom tensor whose optimizer state, failure mode, or initialization could confound the matrix-policy claim.

A posteriori, the completed replacement runs use the same global-rational RLB source for MatrixPolicy and for the non-MatrixPolicy RLB optimizer controls. This makes the evidence easier to interpret. When MatrixPolicy improves over RLB+AdamW, RLB+Lion, RLB+Muon, and other RLB controls under this setting, the difference is attributed to how the optimizer uses RLB structure, not to a local atom path that only some runs used.

C Main Hyperparameter Setting

C.1 Active and Inactive Branches

The main experimental MatrixPolicy rows use the following active surface.

- RLB activation: 'rlb_fused_global_rational'.
- Rational curve: grouped P5/Q4 with SiLU initialization and no local atoms.
- Backbone and rational coefficients: AdamW child optimizer.
- RLB input/output matrices: RationalMatrixPolicyOptimizer child.

- Group-gradient policy: active; see equation 12 through equation 16.
- Positive-gauge rebalance: active; see equation 25 through equation 29.
- Separate rational coefficient optimizer: inactive.
- Coefficient Gram preconditioning: inactive.
- Transport/quotient projection branches: inactive.

This list is part of the method definition. Optional research branches in the code are not part of the main method unless a manifest explicitly enables them.

C.2 Numerical Constants

The main child optimizers use learning rate 3×10^{-4}, minimum learning rate 3×10^{-5}, weight decay 0.10, AdamW $\beta_1 = 0.9$, AdamW $\beta_2 = 0.999$, and $\epsilon = 10^{-8}$ in the main runs. For RLB matrices, the AdamW multiplier is equation 18. The matrix-normalized branch uses strength 0.75, maximum mixture 0.75, momentum 0.95, five Newton-Schulz normalization steps, and the schedule in equation 21. Its final and minimum mixture values are both zero.

The group-gradient policy uses gain strength 0.20, pressure strength 0.10, rational-activity damping 0.20, pressure clip 1.50, activity target 0.05, activity width 0.45, activation window 0.02 to 0.30 of training progress, and final multiplier clip $[0.75, 1.35]$.

The gauge rebalance runs every five optimizer steps. It uses probe range $[-5, 5]$, 129 probe points, target weight 1.0, strength 0.50, activation window 0.03 to 0.35, layer-depth gain 0.15, pressure coefficient 0.25, pressure clip 1.25, rational-activity coefficient 0.10, activity-gain clamp $[0.75, 1.25]$, and maximum absolute log step 0.030.

C.3 Runtime and Failure Accounting

Runtime tables should report clean training-process time from completed JSONL summary records. Queue wait, dependency wait, token-cache construction, extension compilation, launcher overhead, and discarded pre-restart partial attempts are not optimizer runtime. If a completed artifact cannot reconstruct trusted per-row training time after preemption, the row must be rerun or marked as unrepaired rather than silently averaged into the runtime table.

Failures are not excluded from the scientific record. If an optimizer/activation pair diverges or early-stops under the stated protocol, it should be reported as diverged or early-stopped under the same stopping rule. The accounting rule only separates training behavior from systems artifacts such as preempted allocations or known-bad timing nodes.